Assignment Code: DS-AG-019

Neural Network - A Simple Perceptron | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Deep Learning? Briefly describe how it evolved and how it differs from traditional machine learning.

Answer:

Deep Learning is a type of ML using multi-layer neural networks to learn features automatically.
Unlike traditional ML, it does not need manual feature extraction.

Question 2: Explain the basic architecture and functioning of a Perceptron. What are its limitations?

Answer:

A single-layer neural network: input → weights → bias → output.
Limitation: Solves only linear problems, cannot solve XOR.

Question 3: Describe the purpose of activation function in neural networks. Compare Sigmoid, ReLU, and Tanh functions.

Answer:

Sigmoid: (0,1), smooth but vanishing gradient
ReLU: Fast, widely used, may cause dead neurons
Tanh: (-1,1), better than sigmoid but still vanishing gradient

Question 4: What is the difference between Loss function and Cost function in neural networks? Provide examples.

Answer:

Loss: Error for one data point
Cost: Average loss over dataset

Question 5: What is the role of optimizers in neural networks? Compare Gradient Descent, Adam, and RMSprop.

Answer:

GD: Simple, slow
Adam: Fast, adaptive (best)
RMSprop: Good for dynamic data

- **Use NumPy, Matplotlib, and Tensorflow/Keras for implementation.**

Question 6: Write a Python program to implement a single-layer perceptron from scratch using NumPy to solve the logical AND gate.

(*Include your Python code and output in the code box below.*)

Answer:

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])
w = np.zeros(2); b = 0; lr = 0.1
for _ in range(10):
    for i in range(4):
        y_pred = 1 if np.dot(X[i], w)+b >= 0 else 0
        w += lr*(y[i]-y_pred)*X[i]
        b += lr*(y[i]-y_pred)
for i in X:
    print(i, "->", 1 if np.dot(i,w)+b>=0 else 0)
```

Question 7: Implement and visualize Sigmoid, ReLU, and Tanh activation functions using Matplotlib.

(*Include your Python code and output in the code box below.*)

Answer:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10,10,100)
plt.plot(x, 1/(1+np.exp(-x)), label="Sigmoid")
plt.plot(x, np.maximum(0,x), label="ReLU")
plt.plot(x, np.tanh(x), label="Tanh")
plt.legend(); plt.show()
```

Question 8: Use Keras to build and train a simple multilayer neural network on the MNIST digits dataset. Print the training accuracy.

(*Include your Python code and output in the code box below.*)

Answer:

```
from tensorflow.keras import datasets, layers, models

(x_train,y_train),(x_test,y_test)=datasets.mnist.load_data()
x_train,x_test=x_train/255.0,x_test/255.0
model=models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(128,activation='relu'),
    layers.Dense(10,activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train,y_train,epochs=3)
```

Question 9: Visualize the loss and accuracy curves for a neural network model trained on the Fashion MNIST dataset. Interpret the training behavior.

(*Include your Python code and output in the code box below.*)

Answer:

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['accuracy'])
plt.show()
```

Question 10: You are working on a project for a bank that wants to automatically detect fraudulent transactions. The dataset is large, imbalanced, and contains structured features like transaction amount, merchant ID, and customer location. The goal is to classify each transaction as fraudulent or legitimate.

Explain your real-time data science workflow:

- How would you design a deep learning model (perceptron or multilayer NN)?
- Which activation function and loss function would you use, and why?
- How would you train and evaluate the model, considering class imbalance?
- Which optimizer would be suitable, and how would you prevent overfitting?

(*Include your Python code and output in the code box below.*)

Answer:

```
from tensorflow.keras import models, layers

model=models.Sequential([
    layers.Dense(32,activation='relu',input_shape=(10,)),
    layers.Dense(1,activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```